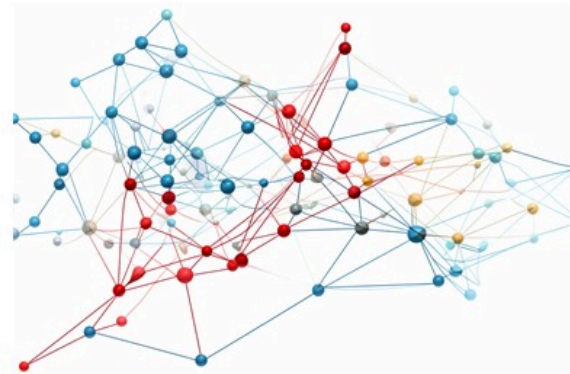




青岛科技大学
Qingdao University of Science & Technology



数据结构与算法分析2

第8章 贪心法1—基本原理

梁宏涛

lht@qust.edu.cn

周二&周四上午

1

第8章—贪心法

提纲 CONTENTS

8.1 贪心法概述

8.2 区间问题

8.5 零钱兑换问题

8.3 背包问题

8.6 哈夫曼编码

8.4 田忌赛马问题

8.7 拟阵*

第8章—贪心法

提纲 CONTENTS

8.1 贪心法概述

8.2 区间问题

8.5 零钱兑换问题

8.3 背包问题

8.6 哈夫曼编码

8.4 田忌赛马问题

8.7 拟阵*

问题引入：

找零问题**1**：拥有面额为**1**、**5**、**10**、**20**、**50**、**100**的纸币，问付出最少的纸币数目完成**n**金额的支付。

示例**1**：

n=8

最少纸币数目**4**，一个**5**元，三个**1**元。

示例**2**：

n=26

最少纸币数**3**，一个**20**元，一个**5**元，一个**1**元。

8.1 贪心法概述

8.1.1 什么是贪心法

- **贪心法 (Greedy Algorithm)** 是一种重要的算法设计策略，用于求解优化问题。
- 贪心法是从问题的某一个**初始状态**出发，通过逐步构造最优解的方法向给定的目标前进，并期望通过这种方法产生出一个**全局最优解**的方法。
- 做出贪心决策的依据称为**贪心准则 (策略)**，并且决策一旦做出，就不可再更改。
- 总之，贪心法总是作出**在当前看来最好的选择**，这个局部最优选择仅依赖以前的决策，但不依赖于以后的决策。

8.1 贪心法概述

8.1.1 什么是贪心法

- **贪心法 (Greedy Algorithm)** 是一种在每个决策阶段都做出在当前看来是最好的选择，期望通过一系列的最优解能得到问题的整体最优解的策略。
- 这种方法是局部最优解的**贪心策略**推向整体最优解。
- 贪心方法并不总是得到整体最优解，但是对于一些问题来说，贪心法得到的却是整体最优解，如最小生成树、霍夫曼编码等问题。

8.1.2 贪心法求解问题具有的性质

1. 最优子结构性质

- 如果问题的最优解所包含的子问题的解也是最优的，就称该问题具有最优子结构性质。
- 通常采用反证法来证明一个问题具有最优子结构性质。

2. 贪心选择性质

- 所谓贪心选择性质是指整体最优解可以通过一系列局部最优选择（即贪心选择）来得到。
- 在证明问题是否具有贪心选择性质时，通常采用数学归纳法证明，先证明第一步贪心选择能够得到整体最优解，再通过归纳步的证明保证每一步贪心选择都能够得到问题的整体最优解。

2. 贪心选择性质

- 是的，确实存在不满足贪心选择性质的问题。
- 找零问题1：拥有面额为1、5、10、20、50、100的纸币，问付出最少的纸币数目完成 n 金额的支付。
- 在这个问题中，贪心选择性质成立。每一步总是选择面额最大的纸币，最后得到的纸币数目肯定是最少的。
- 找零问题2：假如你只拥有面额为1、4、5的硬币，试问如何在给出总额为8的情况下，用最少的硬币数目完成支付。如果我们按照贪心的原则，首先选取一个面额最大的5，再接着选取三个面额为1的硬币，那么总共需要4个硬币。而实际上，最优解应该是选取两个面额为4的硬币，那么只需要两个硬币。

"好的开始是成功的一半。" 这句话译自亚里士多德的名言，**"Well begun is half done."**

表达的思想是抓住当前，将眼前的事情做好，就已经成功了一半，这种立即行动、立即收益的思想符合贪心法思想吗？

- ☒ **A 符合**
- ☐ **B 不符合**

8.1.2 贪心法一般求解过程

- (1) 建立数学模型来描述问题。
- (2) 把求解的问题分成若干个子问题。
- (3) 对每一子问题求解，得到子问题的局部最优解。
- (4) 把子问题的局部最优解合成原问题的最优解。

Alg。	Solution Type Greedy(SType a[],int n){ //贪心算法	
in	数组a和长度n	
out	解向量x	
1	SolutionType x={};	//解向量，初始时空
2	for (int i=0;i<n;i++) {	//执行n步操作
3	xi=Select(a);	//从a中选择一个当前最好的分量
4	if (Feasiable(xi))	//判断xi是否包含在当前解中
5	solution=Union(x,xi);	//将xi分量合并形成x
6	}	//end of for
7	return x;	//返回生成的最优解
8	}	

第8章—贪心法

提纲 CONTENTS

8.1 贪心法概述

8.2 区间问题

8.3 背包问题

8.4 田忌赛马问题

8.5 零钱兑换问题

8.6 哈夫曼编码

8.7 拟阵*

8.2 区间问题

8.2.1 最大兼容区间个数

 **问题描述**：活动安排问题是假设有 n 个活动 $S = (1, 2, \dots, n)$ ，有一个资源，每个活动执行时都要占用该资源，并且该资源任何时刻只能被一个活动所占用，一旦某个活动开始执行，中间不能被打断，直到其执行完毕。每个活动 i 有一个开始时间 s_i 和结束时间 e_i ($s_i < e_i$)，它是一个半开时间区间 $[s_i, e_i)$ 。

假设最早活动执行时间为 0。求一种最优活动安排方案，使得**安排的活动个数最多**。

解

- 假设活动时间的参考原点为0。一个活动 i ($1 \leq i \leq n$) 用一个区间 $[s_i, e_i)$ 表示, 当活动按结束时间 (右端点) 递增排序后, 两个活动 $[s_i, e_i)$ 和 $[s_j, e_j)$ 兼容 (满足 $s_i \geq e_j$ 或 $s_j \geq e_i$) 实际上就是指它们不相交。
- 用数组A存放所有的活动, $A[i].s$ ($1 \leq i \leq n$), 存放活动起始时间, $A[i].e$ 存放活动结束时间。



(a) $s_j \geq e_i$



(b) $s_i \geq e_j$

■ **贪心策略：**每一步总是选择分配这样的一个活动，它能够使得余下的活动的时间最大化即余下活动中兼容活动尽可能多。

例如，对于下表的 $n=11$ 个活动（已按结束时间递增排序）**A**:

i	1	2	3	4	5	6	7	8	9	10	11
开始时间	1	3	0	5	3	5	6	8	8	2	12
结束时间	4	5	6	7	8	9	10	11	12	13	15

产生最大兼容活动集合的过程:

活动1	✓
活动2	×
活动3	×
活动4	✓
活动5	×
活动6	×
活动7	×
活动8	✓
活动9	×
活动10	×
活动11	✓

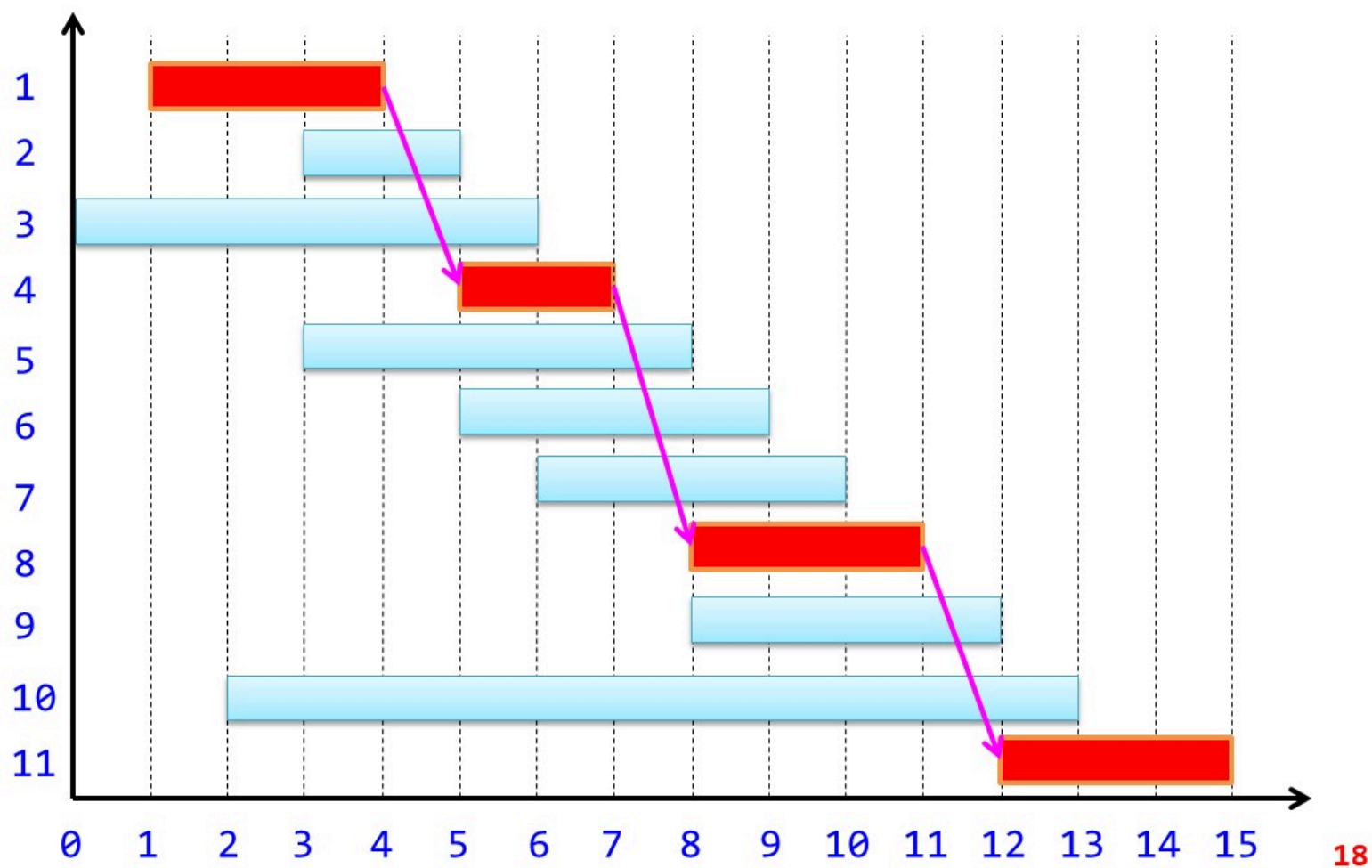
最大兼容活动集合:

活动1 活动4 活动8 活动11

求解结果

17

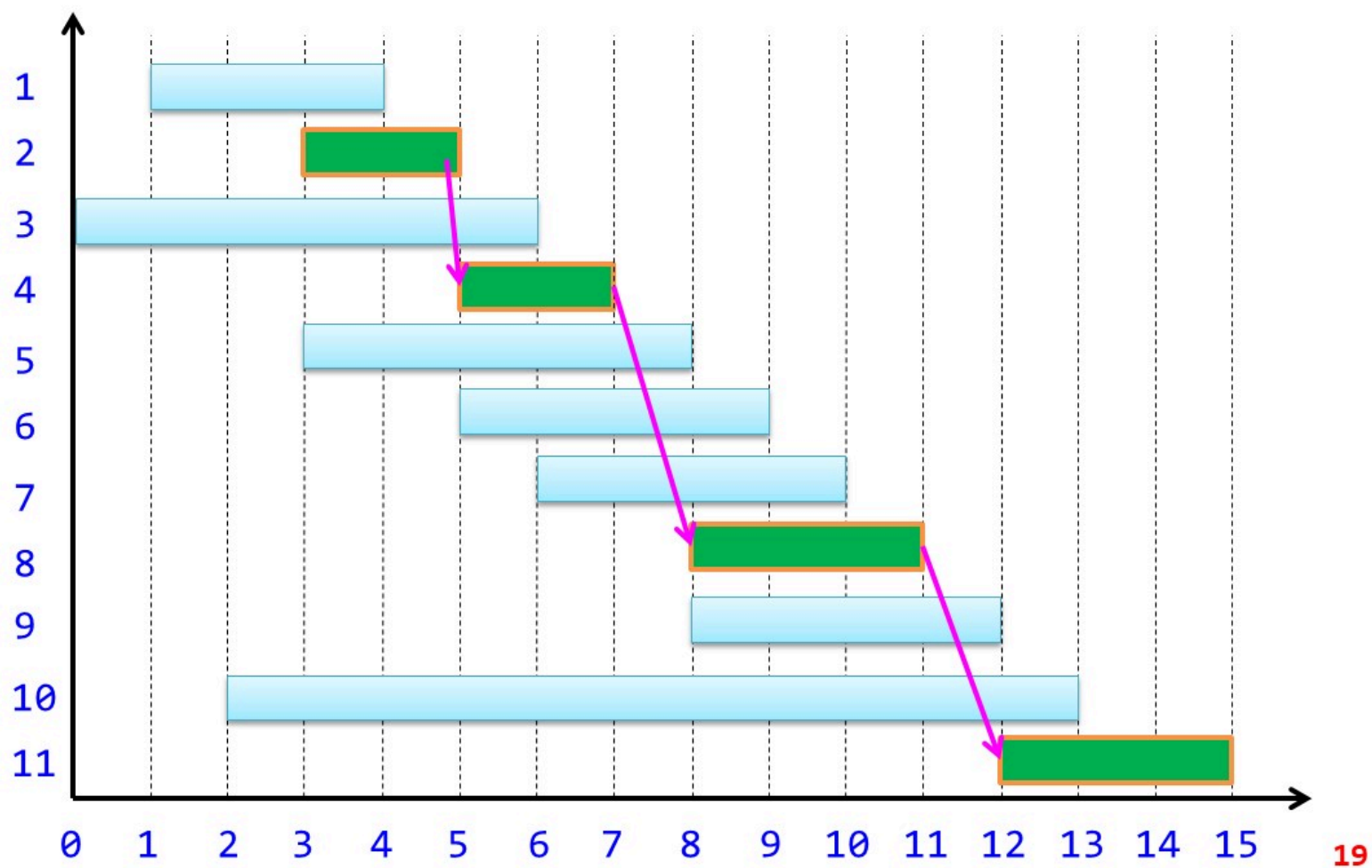
i	1	2	3	4	5	6	7	8	9	10	11
开始时间	1	3	0	5	3	5	6	8	8	2	12
结束时间	4	5	6	7	8	9	10	11	12	13	15



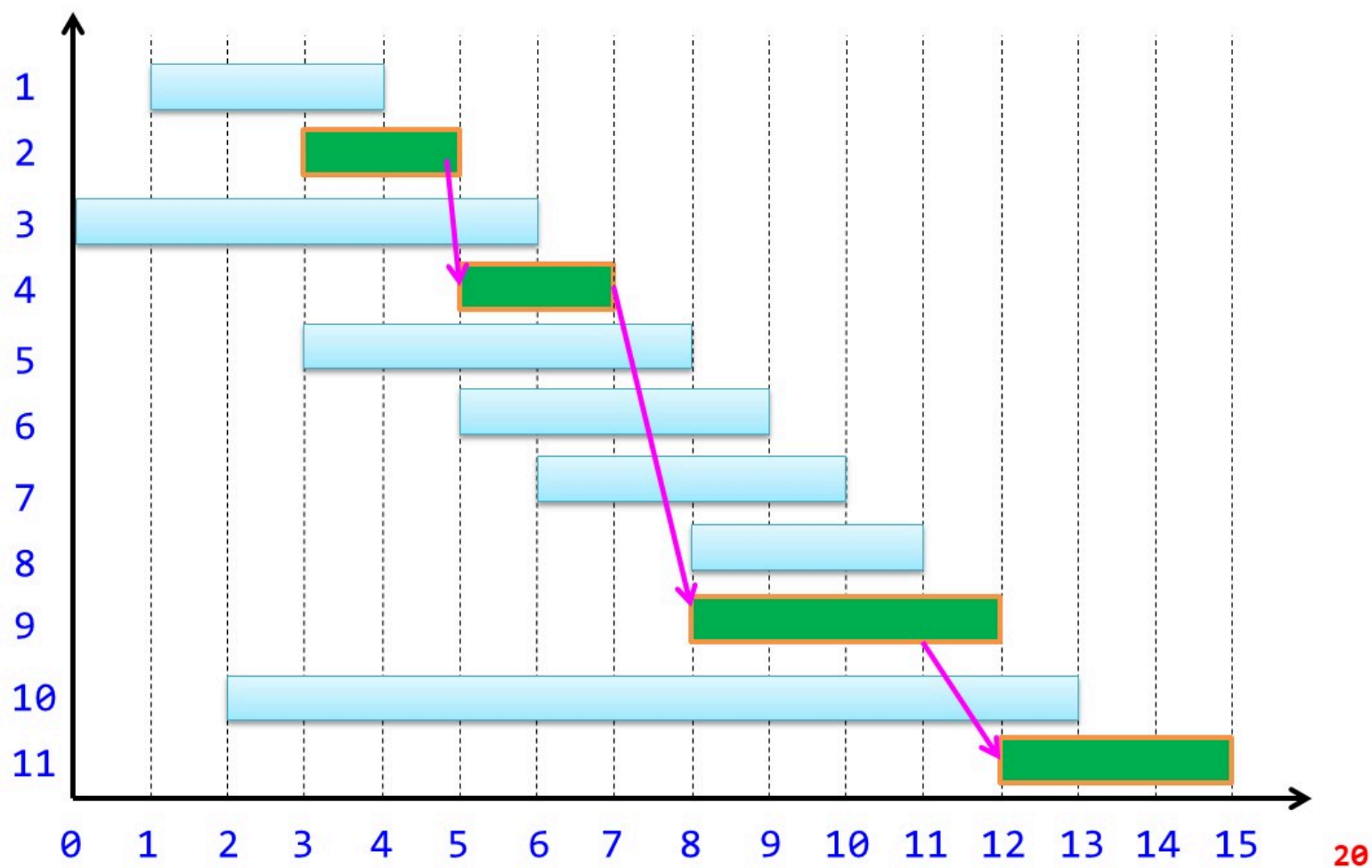
主观题 10分

现有活动 $A=\{(3,5),(2,7),(6,9),(1,2),(4,7)\}$ ，将活动按照结束时间递增排序，并绘制图形，通过贪心法安排最多的活动。

i	1	2	3	4	5	6	7	8	9	10	11
开始时间	1	3	0	5	3	5	6	8	8	2	12
结束时间	4	5	6	7	8	9	10	11	12	13	15



i	1	2	3	4	5	6	7	8	9	10	11
开始时间	1	3	0	5	3	5	6	8	8	2	12
结束时间	4	5	6	7	8	9	10	11	12	13	15




```

1. struct Action {                                //活动的类型
2.     int s;                                       //活动起始时间
3.     int e;                                       //活动结束时间
4.     Action(int s,int e):s(s),e(e) {}
5.
6.     bool operator<(const Action &o) const {    //重载<关系函数
7.         return e<=o.e;                        //用于按活动结束时间递增排序
8.     }
9. };

```

```
10. void greedy(vector<Action>&A) { //求解最大兼容活动子集
11.   int n=A.size();
12.   vector<bool> flag(n,false); //标记选取的活动
13.   sort(A.begin(),A.end());    //将A按活动结束时间递增排序
14.   int preend=A[0].e;           //前一个兼容活动的结束时间
15.   flag[0]=true;
16.   int ans=1;                   //选择的活动个数
```

```

17.  for (int i=1;i<n;i++) {
18.      if (A[i].s>=preend) {           //A[i]与当前区段兼容
19.          flag[i]=true;              //选择A[i]
20.          ans++;
21.          preend=A[i].e;
22.      }
23.  }
24.
25.  printf("求解结果\n");               //输出求解结果
26.  printf("  选取的活动:");
27.  for (int i=0;i<n;i++) {
28.      if (flag[i])
29.          printf("[%d,%d) ",A[i].s,A[i].e);
30.  }
31.  printf("\n  共%d个活动\n",ans);
32. }

```

单选题 1分

 **greedy** 算法分析：算法的主要时间花费在排序上，排序时间为 $O(n\log_2 n)$ ，所以整个算法的时间复杂度为_____

- ☒ A $O(n\log_2 n)$
- ☐ B $O(n^2\log_2 n)$
- ☐ C $O(n^3\log_2 n)$
- ☐ D $O(n!)$

算法证明

最优子结构性质

首先证明总存在一个以活动1开始的最优解:

- ✪ 如果最优解 X 第一个选中的是活动 k ($k \neq 1$), X 中的活动是兼容的, 即 $\forall j \in X$ 有 $e_k \leq s_j$, 又因为 $e_1 \leq e_2 \leq \dots \leq e_n$, 所以 $e_1 \leq e_k \leq s_j$, 设 $Y = X - [s_k, e_k) \cup [s_1, e_1)$, 则 Y 也一定是 A 的一个最优解。这就说明总存在一个以活动1开始的最优解。

证明最优子结构性成立

- **最优子结构性质：**如果 X 是 A 的以活动1开始的最优解，设 $A' = \{i \in A \mid s_i \geq e_1\}$ ，则 $X' = X - \{1\}$ 也是 A' 的最优解。
- 由于 X 是一个最优解， X 中的活动一定是兼容的，则 $X' = X - \{1\}$ 中的活动也一定是兼容的，所以只需要证明 X' 是 A' 的最大兼容活动子集即可。
- 采用反证法，如果 X' 不是 A' 的最大兼容活动子集，则一定有 Y' 是 A' 的兼容活动子集，同时 $|Y'| > |X'|$ 。而 Y' 中一定不含活动1，令 $Y = Y' \cup \{1\}$ ，则 Y 一定是 A 的一个兼容子集，并且 $|Y| > |X|$ ，这样与 X 是 A 的最优解矛盾。

贪心选择性质

- 采用数学归纳法证明满足贪心选择性质：**按照每次选择具有最早结束时间的兼容活动可以得到最优解。**
- 设 X 是 A 的包含活动1的最优解，按 X 中包含的活动兼容个数 k 做归纳证明。
- 当 $k=1$ 时 X 中只有活动1，可以推出 A 中所有活动均不兼容，结论成立。
- 假设 $k < n$ 时结论均成立。
- 当 $k=n$ 时，按照贪心法的过程先从 A 中选择活动1，设 $A' = \{i \in A \mid s_i \geq e_1\}$ ，其最优解必含 $n-1$ 个兼容活动，根据假设可以得到其最优解 X' ，显然 $X = X' \cup \{1\}$ 为 A 的最优解。

27



实例

【例8.2】无重叠区间（LintCode1242★★）。给定一个含有 n 个区间的集合**intervals**，其中**intervals[i]=[si, ei)**，每个区间的类型如下：

```
class Interval {                                //区间的类型
public:
    int start, end;                               //会议开始时间和结束时间
    Interval(int start, int end) { //构造函数
        this->start = start;
        this->end = end;
    }
};
```

可以假设每个区间的右端点一定比左端点大，另外尽管区间[1, 2)和[2, 3)边缘重合，但是它们并未重叠。求需要移除区间的
最小数量，使剩余区间互不重叠。

例如，**intervals**={[1, 2), [2, 3), [3, 4), [1, 3)},
答案是1，移除[1, 3)后剩下的区间没有重叠。

要求设计如下成员函数：

```
int eraseOverlapIntervals(vector<Interval> &intervals) {}
```

28

解法1

- 两个相互不相交的区间就是兼容区间，采用前面活动安排问题的贪心法求出 **intervals** 中最多兼容区间的个数 **ans**,
- 则 **n-ans** 就是使剩余区间互不重叠需要移除区间的最小数量。


```
struct Cmp {  
    bool operator()(const Interval &a, const Interval &b) {  
        return a.end < b.end;    //用于按右端点递增排序  
    }  
};
```

```

1. class Solution {
2. public:
3.     int eraseOverlapIntervals(vector<Interval> &intervals)
4.     {
5.         int n=intervals.size();
6.         if(n<=1) return 0;
7.         sort(intervals.begin(),intervals.end(),Cmp());
8.         int ans=1; //表示兼容区间的个数，初
9.         int preend=intervals[0].end; //存放区间0的右端点
10.        for (int i=1;i<n;i++) { //遍历intervals
11.            if (intervals[i].start>=preend) { //当前区间是兼
12.                ans++;
13.                preend=intervals[i].end;
14.            }
15.        }
16.        return n-ans;

```

15. 执行用时为41ms，内存消耗为4.01MB。

31

解法2

- 也可以用 **ans** 表示需要移除区间的最小数量（初始为 **0**）。
- 若当前区间 **interval[i]** 与前面以 **preend** 结尾的区段相交（**intervals[i].start < preend**），则删除之，置 **ans++**；否则表示不相交，置 **preend = intervals[i].end**。

```
struct Cmp {  
    bool operator()(const Interval &a, const Interval &b) {  
        return a.end < b.end;    //用于按右端点递增排序  
    }  
};
```



```

1. class Solution {
2. public:
3.     int eraseOverlapIntervals(vector<Interval>
        &intervals) {
4.         int n=intervals.size();
5.         if(n<=1) return 0;
6.         sort(intervals.begin(),intervals.end(),Cmp());
7.         int ans=0;                //存放答案
8.         int preend=intervals[0].end;    //存放区间0的右端
           点
9.         for(int i=1;i<n;i++) {        //遍历intervals
10.            if(intervals[i].start<preend) //当前区间不是兼
           容
11.                ans++;
12.            else
13.                preend=intervals[i].end;    //当前区间是兼容
14.        }
15.        return ans;

```

1  执行用时为**41ms**，内存消耗为**5.42MB**。

34

8.2.2 区间合并



【例8.3】 合并区间（LintCode156★）。给出若干闭合区间 intervals:

```
class Interval {                                //区间的类型
public:
    int start, end;                             //会议开始时间和结束时间
    Interval(int start, int end) { //构造函数
        this->start = start;
        this->end = end;
    }
};
```

合并所有重叠的部分，注意[1, 2]和[2, 3]可以合并为[1, 3]。

例如，intervals={ [1, 3], [2, 6], [8, 10], [15, 18] }，输出结果是{ [1, 6], [8, 10], [15, 18] }。

要求设计如下成员函数：

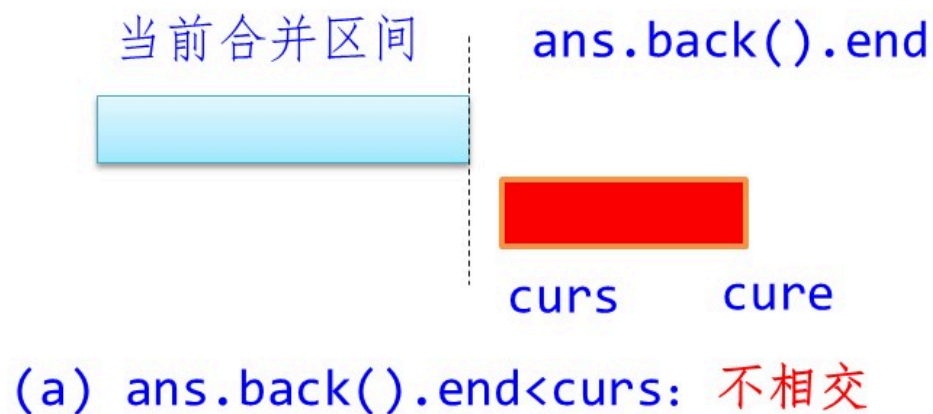
```
vector<Interval> merge(vector<Interval> &intervals) {}
```

35

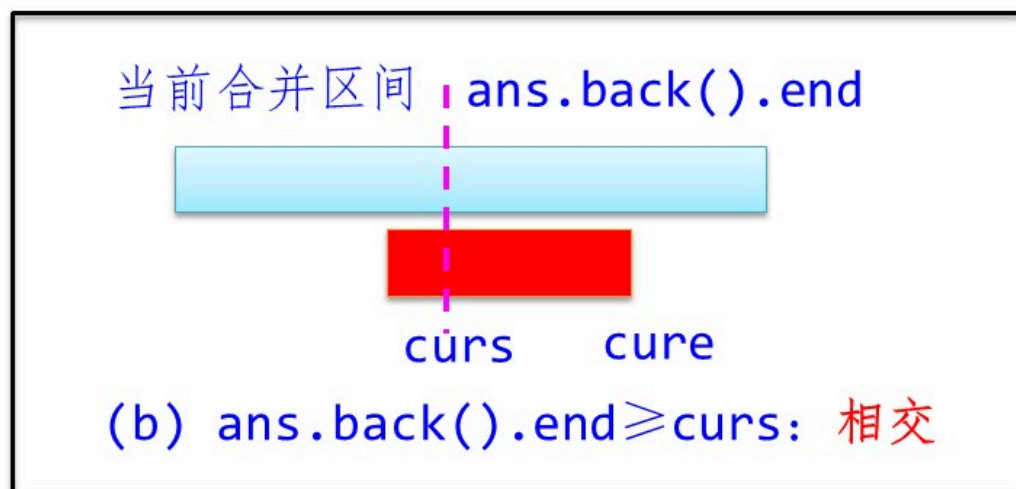
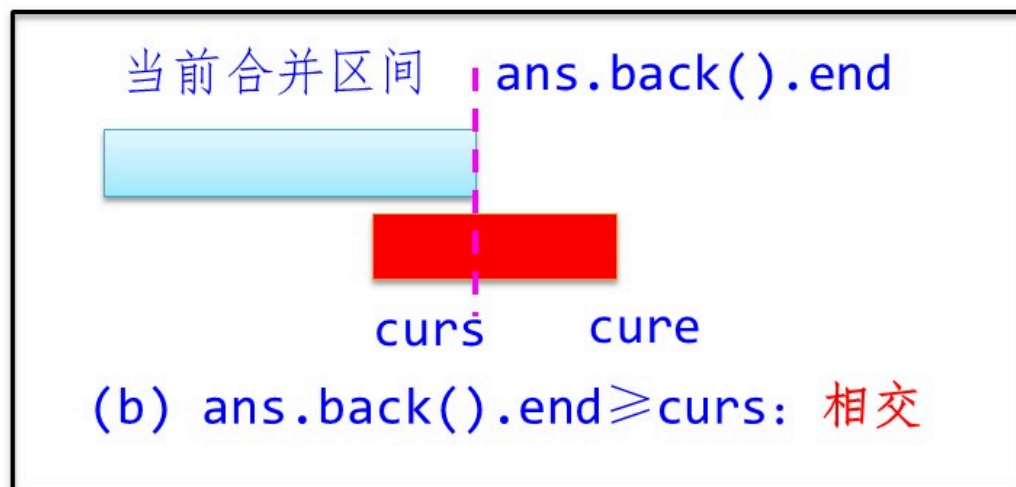
解

- 用 **ans** 存放最终合并的结果（初始为空）。
- 先将全部区间按照左端点递增排序，将区间 **0** 添加到 **ans** 中。
- 用 **i** 从 **1** 开始遍历 **intervals**，设当前区间为 **[curs, cure]**，而当前的合并区间为 **ans** 中末尾区间即 **ans.back()**，分为如下两种情况。

(1) 若 `ans.back().end < curs`, 说明当前区间 *i* 与当前合并区间不相交, 如图(a)所示, 则从当前区间 *i* 开始一个新的合并区间, 即将区间 *i* 添加到 `ans` 中。



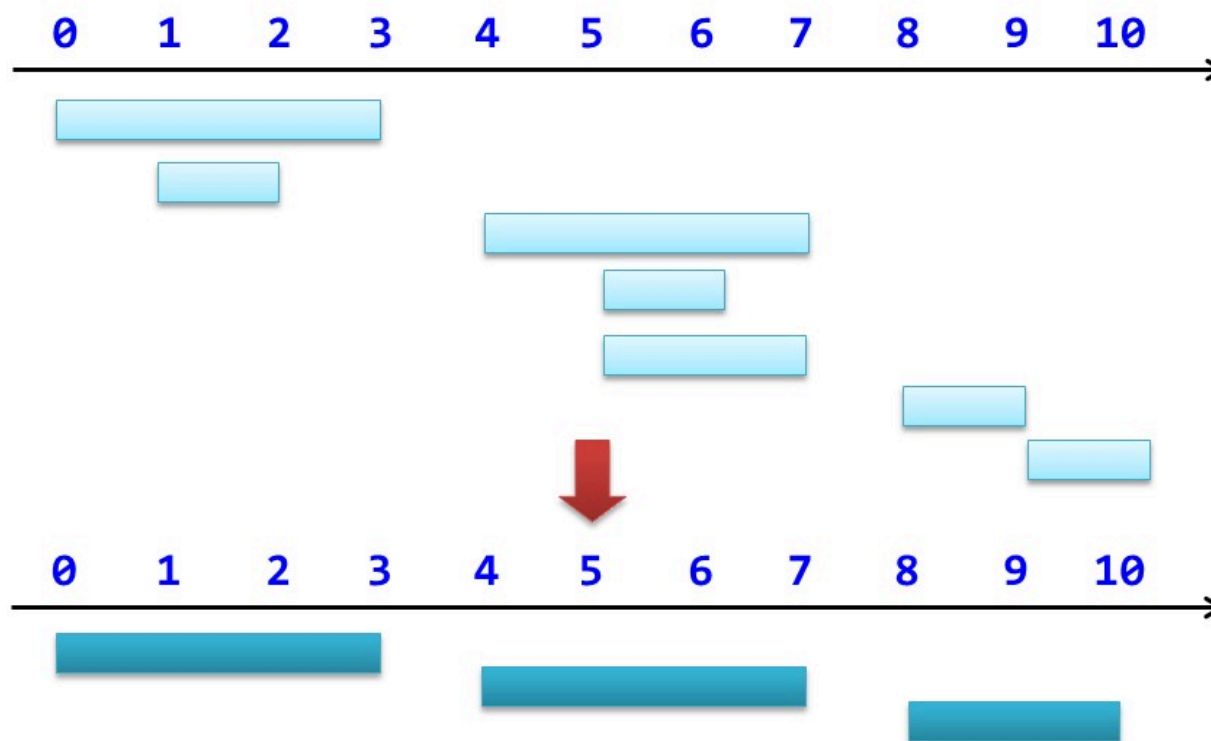
(2) 若 $\text{ans.back().end} \geq \text{curs}$ $\Rightarrow$ 当前区间 i 与当前合并区间相交，如图(b)所示，则将当前区间 i 合并到当前合并区间中，同时更新当前合并区间的右端点为最大值即 $\max(\text{ans.back().end}, \text{cure})$ 。



相交时: `max(ans.back().end, cure)`

■ **贪心策略:** 每次合并时取最大的右端点, 这样就可以合并更多的区间了, 到达整体最优的目的。

- 例如, $\text{intervals}=\{[5, 6], [8, 9], [0, 3], [4, 7], [5, 7], [9, 10], [1, 2]\}$ 。
- 将求按区间左端点递增排序后的结果为 $\text{intervals}=\{[0, 3], [1, 2], [4, 7], [5, 6], [5, 7], [8, 9], [9, 10]\}$ 。
- 按上述过程得到的合并区间是 $\{[0, 3], [4, 7], [9, 10]\}$ 。



```
struct Cmp {  
    bool operator()(const Interval &a, const Interval &b) {  
        return a.start < b.start;    //用于按左端点递增排序  
    }  
};
```



```

1. class Solution {
2. public:
3.     vector<Interval> merge(vector<Interval> &intervals) {
4.         int n=intervals.size();
5.         if (n<=1) return intervals;
6.         sort(intervals.begin(),intervals.end(),Cmp());
7.         vector<Interval> ans;
8.         ans.push_back(intervals[0]);
9.         for (int i=1;i<n;i++) {           //用i遍历intervals
10.            int curs=intervals[i].start; //求当前区间
            [curs,cure)
11.            int cure=intervals[i].end;
12.            if (ans.back().end<curs)      //不相交
13.                ans.push_back({curs,cure});
14.            else                          //相交：合并
15.                ans.back().end=max(ans.back().end,cure);
16.        }
17.        return ans;
18.    }
19. };

```

EXE

执行用时为**61ms**，内存消耗为**5.42MB**。

42

8.2.3 最少资源问题



【例8.4】会议室II（LintCode919★★）。给定一系列的会议时间间隔**intervals**:

```
class Interval {                                //区间的类型
public:
    int start, end;                               //会议开始时间和结束时间

    Interval(int start, int end) { //构造函数
        this->start = start;
        this->end = end;
    }
};
```

每个会议包括起始和结束时间 $\{[s_1, e_1), [s_2, e_2), \dots\}$
($s_i < e_i$)，求所需最小的会议室数量。注意 $[0, 8)$ 和 $[8, 10)$
在8时刻是不冲突的。

例如， $intervals = \{[0, 30), [5, 10), [15, 20)\}$ ，答案是2，最少需要2个会议室， $[0, 30)$ 占用一个会议室， $[5, 10)$ 和 $[15, 20)$ 占用另外一个会议室。

要求设计如下成员函数：

```
int minMeetingRooms(vector<Interval> &intervals) { }
```


解

- 每个会议时间间隔对应一个区间。
- **贪心策略**：优先选择最早开始的会议，同时将尽可能多的与其兼容的含义安排在一个会议室中。
- 先将所有会议按开始时间（左端点）递增排序，用**flag**数组标识会议是否已经安排（初始时所有元素为**false**），顺序处理每个区间。
- 对于尚未安排的最早开始的会议*i*，将其后所有与会议*i*兼容并且未安排的会议*j*安排在一个会议室中，置**flag[j]**为**true**。


```
struct Cmp {  
    bool operator()(const Interval &a, const Interval &b) {  
        return a.start < b.start;           //用于按start递增排序  
    }  
};
```

```
class Solution {  
public:  
    int minMeetingRooms(vector<Interval> &intervals) {  
        int n=intervals.size();  
        vector<bool> flag(n,false);  
        sort(intervals.begin(),intervals.end(),Cmp());  
        int ans=0;
```

```

for(int i=0;i<n;i++) {
    if(!flag[i]) {                //会议i安排在一个新会议室中
        ans++;                    //会议室个数增加1
        int preend=intervals[i].end;
        for(int j=i+1;j<n;j++) {
            if(!flag[j] && intervals[j].start>=preend) {
                preend=intervals[j].end; //会议j安排在会
            }
        }
    }
}
return ans;
};

```

会议室i

EXE

执行用时为**41ms**，内存消耗为**5.42MB**。

算法优化

- 利用优先队列（小根堆）提高性能，因为一旦某个会议安排会议室后就不必再处理，所以用优先队列`minpq`保存每个会议室中最大会议结束时间。对于当前会议`intervals[i]`:
 - 若 `intervals[i].start ≥ minpq.top()`，说明会议`i`与堆顶的会议兼容，将会议`i`安排在对应的会议室中，并用会议`i`的结束时间替代之。
 - 若 `intervals[i].start < minpq.top()`，说明会议`i`与优先队列中任何会议都不兼容，只能安排一个新会议室，将会议`i`的结束时间进队。
- 最后`minpq`中的元素个数即为安排的会议室个数。


```
struct Cmp {  
    bool operator()(const Interval &a, const Interval &b) {  
        return a.start < b.start;    //用于按start递增排序  
    }  
};
```

```

class Solution {
public:
    int minMeetingRooms(vector<Interval> &intervals) {
        priority_queue<int,vector<int>,greater<int>> minpq;
        sort(intervals.begin(),intervals.end(),Cmp());
        minpq.push(intervals[0].end);

        for (int i=1;i<intervals.size();i++) {
            if (intervals[i].start>=minpq.top()) //兼容时出队
                minpq.pop();
            minpq.push(intervals[i].end);
        }
        return minpq.size();
    }
};

```

EXE 执行用时为**40ms**，内存消耗为**5.45MB**。

第8章—贪心法

提纲 CONTENTS

8.1 贪心法概述

8.2 区间问题

8.3 背包问题

8.4 田忌赛马问题

8.5 零钱兑换问题

8.6 哈夫曼编码

8.7 拟阵*

8.3 背包问题

- 有 n 个编号为 $0 \sim n-1$ 的物品，重量为 $w=\{w_0, w_1, \dots, w_{n-1}\}$ ，价值为 $v=\{v_0, v_1, \dots, v_{n-1}\}$ ，给定一个容量为 W 的背包。
- 从这些物品中选取全部或者部分物品装入该背包中，找到选中物品不仅能够放到背包中而且价值最大的方案。
- 与0/1背包问题的区别是这里的每个物品可以取一部分装入背包。

物品编号no	0	1	2	3	4
w	10	20	30	40	50
v	20	30	66	40	60

$W=100$

解

■ 贪心策略：每次选择单位重量价值最大的物品。

序号 <i>i</i>	0	1	2	3	4
物品编号no	2	0	1	4	3
<i>w</i>	30	10	20	50	40
<i>v</i>	66	20	30	60	40
<i>v/w</i>	2.2	2.0	1.5	1.2	1.0

$x[0]=1$
 $bestv=66$
 $rw=rw-w[0]=70$

$x[1]=1$
 $bestv=66+20=86$
 $rw=rw-w[1]=60$

$x[2]=1$
 $bestv=86+30=116$
 $rw=rw-w[2]=40$

$x[3]=0.8$
 $bestv=116+0.8*60$
 $=164$

```

1. void knap(vector<Goods>&g,int W) { //求解背包问题
2.     sort(g.begin(),g.end()); //按v/w递减排序
3.     int n=g.size();
4.     vector<double> x(n,0.0); //存放最优解向量
5.     int bestv=0; //存放最大价值,初始为0
6.     double rw=W; //背包中能装入的余下重量
7.     int i=0;
8.     while (i<n && g[i].w<rw) { //物品i能够全装入
9.         x[i]=1; //装入物品i
10.        rw-=g[i].w; //减少背包的余下重量
11.        bestv+=g[i].v; //累计总价值
12.        i++; //继续循环
13.    }

```

```

1.   if (i<n && rw>0) {                               //当余下重量大于0
2.       x[i]=rw/g[i].w;                               //将物品i的一部分装入
3.       bestv+=x[i]*g[i].v;                           //累计总价值
4.   }
5.   printf("最优解\n");                               //输出结果
6.   for (int j=0;j<n;j++) {
7.       if(x[j]==1)
8.           printf("  选择物品%d[%d,%d]百分之百\n",
9.               g[j].no,g[j].w,g[j].v);
10.      else if(x[j]>0)
11.          printf("  选择物品%d[%d,%d]为百分之%.1f\n",
12.              g[j].no,g[j].w,g[j].v,x[j]*100);
13.  }
14.  printf("  总价值=%d\n",bestv);
15. }

```

算法分析：排序算法 `sort()` 的时间复杂性为 $O(n\log_2 n)$ ，while 循环的时间为 $O(n)$ ，所以算法的时间复杂度为 $O(n\log_2 n)$ 。



【例8.5】分饼干（LintCode1230★）。有 n

($1 \leq n \leq 30000$) 个孩子，现在给孩子们发一些小饼干，每个孩子最多只能给一块饼干。每个孩子 i 有一个胃口值 $g[i]$

($1 \leq g[i] \leq 2^{31}-1$)，这是能满足胃口的最小饼干尺寸，共有 m ($1 \leq m \leq 30000$) 块饼干，每块饼干 j 有一个尺寸 $s[j]$ ($1 \leq s[j] \leq 2^{31}-1$)。如果 $g[i] \leq s[j]$ ，那么将饼干 j 分发给孩子 i 时该孩子会得到满足。分发的目标是尽可能满足越多数量的孩子，设计一个算法求这个最大数值。

例如， $g=\{1, 2, 3\}$ ， $s=\{1, 1\}$ ，尽管有两块小饼干，由于尺寸都是1，只能让胃口值是1的孩子满足，所以结果是1。

要求设计如下成员函数：

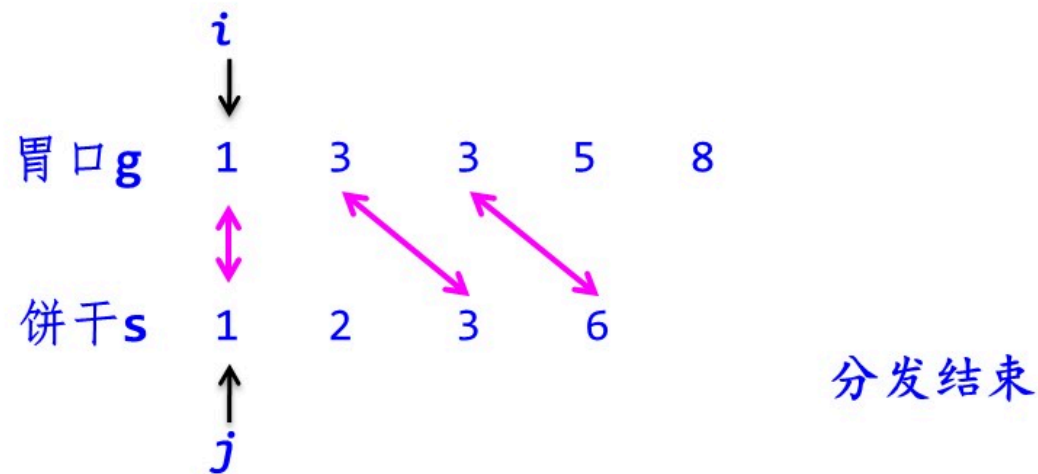
```
int findContentChildren(vector<int> &g, vector<int> &s) {}
```


解

- 求得到满足的最多孩子数量，所以是一个求最优解问题。
- 贪心策略：胃口值越小越容易满足，优先分发给胃口值小的孩子。为此将 g 递增排序。
- 对于每个 $g[i]$ ，为其分发恰好满足的最小尺寸的饼干 j 即 $\min\{j | g[i] \leq s[j]\}$ 。为此将 s 也递增排序。

- 用 **ans** 表示得到满足的最多孩子数量（初始为 0）即最优解，**i** 从 0 开始遍历 **g**，**j** 从 0 开始在 **s** 中查找：
 - ① **$g[i] \leq s[j]$** ，说明为孩子 **i** 分发饼干 **j** 得到满足，则将饼干 **j** 分发给孩子 **i**，执行 **ans++**，同时执行 **i++**，**j++** 继续为下一个孩子分发合适的饼干。
 - ② 否则，孩子 **i** 得不到满足，执行 **j++** 继续为其查找更大尺寸的饼干。
- 最后的 **ans** 就是答案。

例如, $g=\{3, 1, 5, 3, 8\}$, $s=\{6, 1, 3, 2\}$, 排序后
 $g=\{1, 3, 3, 5, 8\}$, $s=\{1, 2, 3, 6\}$, 分发饼干的过程
如下图所示, 结果 $ans=3$ 。



```

class Solution {
public:
    int findContentChildren(vector<int> &g,vector<int> &s) {
        int m=g.size();
        int n=s.size();
        sort(g.begin(),g.end());           //默认为递增排序
        sort(s.begin(),s.end());
        int ans=0;
        for (int i=0,j=0; i<m && j<n;j++) {
            if (g[i]<=s[j]) {
                i++;
                ans++;
            }
        }
        return ans;
    }
};

```


EXE 执行用时为**104ms**，内存消耗为**5.48MB**。



如果将贪心选择策略改为每个孩子 i 分发得到满足的饼干 j （不一定是最小尺寸的饼干），结果是否正确呢？

